

Wallwright and Hiperwall

A head to head against the closest commercial equivalent.

Prepared for the Wallwright exhibit · 24 August 2026 · **VERIFIED** marks a claim a cited source states. **UNVERIFIED** marks inference or something not checked. Hiperwall's platform details are quoted from Hiperwall's own documentation.

Why Hiperwall is the comparison

Most of this market rules itself out on one requirement: the exhibit runs on a Windows PC, and much of the field is Linux appliances streaming video to a decoder behind each display. Hiperwall is the exception, and the closest thing to Wallwright that can be bought. It runs on Windows, on ordinary PCs, and it renders web pages with a real browser engine rather than a video stream. **VERIFIED**

That makes it the useful comparison. The two agree on the architecture, so the question is not which approach is right; it is what the differences cost.

What each one is

Hiperwall

Software on ordinary Windows machines. "All Hiperwall PCs require Windows 10 or 11 (32 or 64 bit) and Gigabit Ethernet", and it "runs on standard PCs connected by standard Ethernet". Each display is driven by a PC running HiperView, and one PC can drive several displays. **VERIFIED**

Crucially, its web support is not a screen capture. HiperSource Browser is described as a real web browser based on the Chromium engine, providing a direct connection between web content and the wall and **bypassing the screen capture approach**. Keyboard and mouse control of a source comes through a separate HiperKVM licence. **VERIFIED**

Wallwright

A single application on the show PC. Each panel is a real browser view composited onto the wall's output, and input reaches it the way it reaches a browser tab. **VERIFIED**

So the two agree on the hard architectural question. Both reject streaming, both put a real browser engine on the wall, and both run on Windows. The differences are everywhere else.

The differences that matter

	HIPERWALL	WALLWRIGHT
Operating system	Windows 10 or 11 VERIFIED	Windows, and macOS for development VERIFIED

	HIPERWALL	WALLWRIGHT
Rendering	Real Chromium, not screen capture VERIFIED	Real Chromium, composited natively VERIFIED
Machines	A controller plus one or more HiperView PCs. Whether one PC can be both was not confirmed. UNVERIFIED	One: the show PC VERIFIED
Licence	Per display, and each browser tab consumes one HiperSource licence . A four panel montage needs four. VERIFIED	None
Interaction	Through a HiperKVM licence VERIFIED	Native, no extra layer VERIFIED
Failover	HiperFailSafe, a second controller VERIFIED	None, and on one PC cannot have one

The licensing shape is the sharpest practical difference

Hiperwall licenses per display, and every browser tab consumes a HiperSource licence. The exhibit's whole idea is a montage of live pages, and the editor now lets someone add panels at the wall. Under that model, the natural act of adding a panel has a purchase order attached to it. Wallwright's panel count is free, which is not a small difference for an exhibit whose layout is meant to be changed.

Where Hiperwall wins

- **Scale.** Many displays, many nodes, one wall or several. Wallwright drives one output.
- **Redundancy.** HiperFailSafe runs a second controller. **VERIFIED**
- **Maturity and support.** A maintained product with an external API, outliving whoever wrote the exhibit. **VERIFIED**
- **It has done this before.** Real installations, real control rooms. Wallwright has never run a show.

Where Wallwright wins for this exhibit

- **One machine, no licence, no per-panel cost.** It runs on the PC that is already driving the wall.
- **Interaction without a KVM layer.** Clicking and typing are native, not a licensed feature.
- **Arranging the montage on the live wall,** against the real dashboards at the real resolution, rather than authoring layouts in a console. **UNVERIFIED THAT HIPERWALL CANNOT DO THIS**
- **Several panels behind one login.** Four dashboards from one SSO-protected application share a session instead of needing four sign-ins. No vendor documents an equivalent. **UNVERIFIED**

- **Measured against this exact job.** Session behaviour, memory, and the interaction model were tested for this exhibit, not inferred from a datasheet.

Cost, honestly

Hiperwall does not publish pricing, so the comparison cannot be made in numbers. **UNVERIFIED** What can be said is the shape: a per display licence, plus a HiperSource licence per browser tab, plus HiperKVM for interaction, plus HiperFailSafe if redundancy is wanted. Against that, Wallwright costs the engineering already spent and nothing recurring.

The costs that do not appear on either invoice are the ones worth weighing. Buying brings support and someone else's testing. Building brings an exact fit and a dependency on whoever maintains it.

Recommendation

For this exhibit as scoped, Wallwright is the better fit, mainly because the licensing model taxes the thing the exhibit is built to do, and because a single wall of web pages does not need a controller and display nodes.

Hiperwall becomes the right answer if any of these turn out to be true:

1. It grows to several walls, or many displays.
2. Unattended reliability with failover becomes a hard requirement.
3. Honeywell will not accept an unsigned in-house application, but will accept a commercial product. Worth asking early, because it decides everything else.
4. **Overlay compositing fails on Windows.** This is the one unverified assumption underneath Wallwright. Hiperwall has clearly solved the equivalent problem, and if ours does not hold, buying gets much more attractive.

What is still unverified

- Hiperwall pricing, in any form.
- Whether one Windows PC can serve as both HiperController and HiperView node, which decides whether this is a one machine or two machine deployment.
- Whether either product handles a real Honeywell SSO flow. Hiperwall's material mentions users logging in to restricted sites, which is the closest anyone comes, but it is not a commitment.

VERIFIED AS A CLAIM, NOT AS BEHAVIOUR

- Wallwright's overlay compositing on Windows. An automated check was attempted. The build machine has a connected desktop, but the runner runs as a service in Windows session 0, which is isolated from it, so it cannot photograph the screen. The fix is to run the runner in the console session rather than as a service; short of that, someone runs the app on a Windows machine and looks at it. It remains the largest open risk in the project.

Sources

1. Hiperwall system requirements, Windows 10 or 11 on standard PCs. <https://serviceportal.sharpnecdisplays.eu/p/eeme/en/products/software/details/systemrequirements/t/Software/Displays-and-Projectors/rp/HIPERWALL-NEW.xhtml>
2. Hiperwall, HiperSource Browser: real Chromium, bypasses screen capture. <https://hiperwall.com/browser/>
3. Hiperwall, HiperView display nodes. <https://hiperwall.com/technology-stack/hiperview/>
4. Hiperwall, control room video walls: sources, HiperKVM, licensing per tab. <https://connect.hiperwall.com/control-room-video-walls>
5. Hiperwall, Premium Suite and HiperFailSafe. <https://www.hiperwall.com/hiperwall-premium-suite>
6. Hiperwall, HiperController and the external API. <https://hiperwall.com/technology-stack/hipercontroller/>
7. Wallwright design rationale: SPEC.md in the repository.
8. Wallwright measurements: docs/validation.md in the repository.